

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Modelling Comparative Analysis
Assignment

COURSE NAME: **ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01**
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)*

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A
Modelling Comparative Analysis Assignment

Code of Conduct:

I SHAM S. (192325076) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage (%)	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Domain Knowledge and Fact Identification	10%	
3. Production Rule Modelling	15%	
4. Propositional Logic Modelling	10%	
5. First-Order Logic Modelling	15%	
6. Prolog Modelling and Implementation	15%	
7. Forward and Backward Chaining Demonstration	10%	
8. Comparative Analysis of Modelling Approaches	10%	
9. Results, Discussion and Model Selection	3%	

10. Documentation, GitHub Repository and Presentation	2%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Question / Problem Statement
1	Automobile Fault Diagnosis: An automobile service center wants to identify vehicle faults based on symptoms such as engine overheating, starting failure, abnormal noise, low mileage, and warning indicators. Model the problem using production rules, propositional logic, first-order logic, and Prolog. Demonstrate forward and backward chaining and compare the approaches based on expressiveness, inference, complexity, and suitability.
2	Industrial Machine Fault Diagnosis: A manufacturing plant needs to identify machine faults using observations such as abnormal vibration, high temperature, unusual noise, pressure variation, and reduced production speed. Develop different knowledge representation models and compare their ability to represent machine components, relationships, rules, and diagnostic conclusions.
3	Cybersecurity Threat Detection: A security operations center wants to identify threats from repeated login failures, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Model the threat-detection problem using production rules, propositional logic, first-order logic, and Prolog, and compare forward and backward reasoning for deriving security conclusions.
4	Healthcare Diagnosis: A healthcare system needs to identify possible diseases from symptoms such as fever, cough, fatigue, breathing difficulty, and body pain. Construct comparative knowledge models using production rules, propositional logic, first-order logic, and Prolog. Analyze which representation provides better expressiveness and reasoning capability for diagnosis.

5	Agricultural Crop Disease Diagnosis: Farmers need assistance in identifying crop diseases based on leaf color, spots, wilting, soil conditions, humidity, and pest observations. Represent the agricultural knowledge using different modelling approaches and compare their effectiveness in representing relationships, applying rules, and deriving recommendations.
6	Banking Loan Eligibility: A bank wants to determine loan eligibility using income, credit score, employment status, existing loans, and repayment history. Model the decision-making process using production rules, propositional logic, first-order logic, and Prolog, and compare the reasoning capabilities of each approach.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Problem Statement & Objectives	Describe the selected real-world problem, domain, decision-making requirement, and objectives of the modelling exercise.
2	Domain Knowledge & Facts	Identify entities, attributes, facts, relationships, conditions, and conclusions relevant to the selected problem.
3	Production Rule Model	Represent the domain using IF–THEN production rules and explain how the rules derive conclusions.
4	Propositional Logic Model	Represent the same problem using propositions, logical connectives, and suitable inference statements.
5	First-Order Logic Model	Represent entities, relationships, properties, and rules using predicates and quantifiers.
6	Prolog Model & Implementation	Convert the knowledge into Prolog facts, rules, predicates, and queries. Include relevant code and outputs.
7	Forward & Backward Chaining	Demonstrate both reasoning mechanisms using selected facts/goals and provide the inference sequence.
8	Comparative Analysis (Table structure given)	Compare the models based on expressiveness, inference mechanism, complexity, scalability, flexibility, explainability, and suitability.

9	Results, Discussion & Recommended Model	Analyze the results and justify which approach is most appropriate for the selected real-world problem. Discuss limitations and possible improvements.
10	Conclusion, References & GitHub	Summarize findings, provide references, and include the GitHub repository link containing Prolog code, models, test cases, and supporting materials.

1. Problem Statement & Objectives

A Security Operations Center (SOC) needs a compact rule-based model that can classify possible cybersecurity threats from observable events. The selected assessment scenario specifies repeated login failures, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic as the main observations. The model must represent the same knowledge using production rules, propositional logic, first-order logic, and Prolog, and compare forward and backward reasoning.

Objectives:

Represent security observations as domain facts.

Construct IF–THEN production rules for threat classification.

Express the same rules using propositional logic.

Represent users, events, and threat relationships using first-order predicates.

Implement the knowledge base and reasoning mechanisms in SWI-Prolog.

Demonstrate forward chaining and backward chaining with inference traces.

Compare the four modelling approaches and select the most suitable model.

2. Domain Knowledge & Facts

Entities:

User: Alice, Bob, Charlie, David.

Security events: repeated login failures, unusual login location, privilege escalation, suspicious file access, abnormal network traffic.

Conclusions: brute-force attack, credential compromise, account compromise, malware suspected, high-risk incident, major security threat.

Sample event facts:

User	Observed events
Alice	Repeated login failures; unusual login location; privilege escalation; suspicious file access; abnormal network traffic
Bob	Repeated login failures; unusual login location
Charlie	Suspicious file access; abnormal network traffic
David	Unusual login location

3. Production Rule Model

IF repeated login failures THEN brute-force attack.

IF repeated login failures AND unusual login location THEN credential compromise.

IF unusual login location AND privilege escalation THEN account compromise.

IF suspicious file access AND abnormal network traffic THEN malware suspected.

IF account compromise AND suspicious file access THEN high-risk incident.

IF credential compromise AND abnormal network traffic THEN major security threat.

The rules are monotonic: once an observation or derived conclusion is available, it remains available during the forward-chaining cycle. The model intentionally uses positive conditions so that the inference trace remains easy to inspect.

4. Propositional Logic Model

For one analysed user, let:

R = repeated login failures

L = unusual login location

P = privilege escalation

F = suspicious file access

N = abnormal network traffic

B = brute-force attack

C = credential compromise

A = account compromise

M = malware suspected

H = high-risk incident

T = major security threat

Logical implications:

$R \rightarrow B$

$(R \wedge L) \rightarrow C$

$(L \wedge P) \rightarrow A$

$(F \wedge N) \rightarrow M$

$(A \wedge F) \rightarrow H$

$(C \wedge N) \rightarrow T$

Example for Alice: R, L, P, F and N are true. Therefore B, C, A and M can be inferred; then H and T follow from the derived conclusions.

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.9278]
(c) Microsoft Corporation. All rights reserved.

C:\Users\LENOVO\Downloads\MEDI-EXPERT-Cybersecurity-C05-AT3>"C:\Program Files\swipl\bin\swipl.exe" cyber_threat_detection.pl
Warning: c:/users/lenovo/downloads/medi-expert-cybersecurity-co5-at3/cyber_threat_detection.pl:101:
Warning: Singleton variables: [U]
Warning: c:/users/lenovo/downloads/medi-expert-cybersecurity-co5-at3/cyber_threat_detection.pl:114:
Warning: Singleton variables: [U]
Warning: c:/users/lenovo/downloads/medi-expert-cybersecurity-co5-at3/cyber_threat_detection.pl:217:
Warning: Singleton variables: [U]
Cybersecurity Threat Detection model loaded. Run start.
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

1 ?- start.

Action (h for help) ? Unknown option (h for help)
Action (h for help) ? Unknown option (h for help)
Action (h for help) ? Unknown option (h for help)
Action (h for help) ? Unknown option (h for help)
Action (h for help) ? Unknown option (h for help)
Action (h for help) ? Unknown option (h for help)
Action (h for help) ? Unknown option (h for help)
Action (h for help) ? Unknown option (h for help)
Action (h for help) ? exit (status 4)

C:\Users\LENOVO\Downloads\MEDI-EXPERT-Cybersecurity-C05-AT3>run_demo.
'run_demo.' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\LENOVO\Downloads\MEDI-EXPERT-Cybersecurity-C05-AT3>start.

C:\Users\LENOVO\Downloads\MEDI-EXPERT-Cybersecurity-C05-AT3>run_demo.
'run_demo.' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\LENOVO\Downloads\MEDI-EXPERT-Cybersecurity-C05-AT3>run_tests.
```

5. First-Order Logic Model

Predicates:

RepeatedFailures(u)

UnusualLocation(u)

PrivilegeEscalation(u)

SuspiciousFileAccess(u)

AbnormalTraffic(u)

BruteForce(u)

CredentialCompromise(u)

AccountCompromise(u)

MalwareSuspected(u)

HighRiskIncident(u)

MajorThreat(u)

Universal rules:

$\forall u \text{ (RepeatedFailures}(u) \rightarrow \text{BruteForce}(u))$

$\forall u \text{ ((RepeatedFailures}(u) \wedge \text{UnusualLocation}(u)) \rightarrow \text{CredentialCompromise}(u))$

$\forall u \text{ ((UnusualLocation}(u) \wedge \text{PrivilegeEscalation}(u)) \rightarrow \text{AccountCompromise}(u))$

$\forall u \text{ ((SuspiciousFileAccess}(u) \wedge \text{AbnormalTraffic}(u)) \rightarrow \text{MalwareSuspected}(u))$

$\forall u \text{ ((AccountCompromise}(u) \wedge \text{SuspiciousFileAccess}(u)) \rightarrow \text{HighRiskIncident}(u))$

$\forall u \text{ ((CredentialCompromise}(u) \wedge \text{AbnormalTraffic}(u)) \rightarrow \text{MajorThreat}(u))$

The first-order representation is more expressive than a purely propositional model because the same rule can be stated once for every user rather than creating a separate proposition for each user.

6. Prolog Model & Implementation

The supplied Prolog implementation uses SWI-Prolog. The assessment asks for Prolog facts, rules, predicates, queries and outputs. The implementation keeps the knowledge in rule/3 clauses and provides separate procedures for forward chaining, backward diagnosis, explanation, demonstrations and test cases.

Core facts:

event(alice, repeated_login_failures).

event(alice, unusual_login_location).

event(alice, privilege_escalation).

event(alice, suspicious_file_access).

event(alice, abnormal_network_traffic).

Example rule:

rule(credential_compromise, U, [repeated_login_failures(U), unusual_login_location(U)]).

Example backward query:

backward_diagnoses(alice, Threats).

Expected result includes brute_force_attack, credential_compromise, account_compromise, malware_suspected, high_risk_incident and major_security_threat.

7. Forward & Backward Chaining

7.1 Forward chaining – Alice

Initial facts:

[repeated_login_failures, unusual_login_location, privilege_escalation, suspicious_file_access, abnormal_network_traffic]

Step 1: brute_force_attack, credential_compromise, account_compromise and malware_suspected are derived.

Step 2: high_risk_incident and major_security_threat are derived.

The engine stops when no rule produces a new conclusion.

7.2 Backward chaining – major_security_threat(Alice)

Goal: major_security_threat(alice).

Rule requires credential_compromise(alice) and abnormal_network_traffic(alice).

credential_compromise(alice) requires repeated_login_failures(alice) and unusual_login_location(alice).

All required observations are facts, so credential_compromise(alice) succeeds.

abnormal_network_traffic(alice) is a fact.

Therefore major_security_threat(alice) succeeds.

Backward chaining is goal-directed: it starts from the requested conclusion and searches only the rule dependencies needed to prove that goal.

8. Comparative Analysis

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge representation	IF–THEN rules	Boolean propositions	Predicates, variables, quantifiers	Facts + rules + executable queries
Expressiveness	Moderate	Low–moderate	High	High
Rule representation	Natural IF–THEN form	Implications	Universal implications	Horn-clause style rules
Inference mechanism	Rule firing	Logical entailment	Logical entailment with variables	Resolution/unification with search
Forward chaining	Natural	Possible but external mechanism needed	Possible	Can be implemented explicitly
Backward chaining	Possible	Possible but less direct	Natural for goals	Natural through rule/query evaluation
Relationships	Limited	Weak	Strong	Strong
Scalability	Depends on rule engine	Poor for many entities	Better	Good for structured rule bases
Explainability	High	High	High	High when traces/explanations are added
Ease of implementation	Easy	Moderate	Moderate–high	High for rule-based systems
Suitability here	Good	Useful for simple cases	Very good	Best overall

9. Results, Discussion & Recommended Model

The four representations can encode the selected cybersecurity knowledge, but they differ in how directly they support a reusable rule base. Propositional logic is clear for a small fixed set of events and a single scenario, but it becomes repetitive when many users and events must be represented. First-order logic improves expressiveness through variables and relationships. Production rules are highly readable for domain experts. Prolog combines declarative knowledge representation with executable logical inference, unification and backtracking, making it the most suitable model for this assessment.

Recommended model: Prolog, supported by production-rule knowledge. It provides a compact implementation, reusable variables for multiple users, goal-directed queries, and an explicit forward-chaining procedure for comparison.

Limitations: the model uses a small fixed event vocabulary, deterministic positive rules, no event timestamps or severity scores, no probabilistic uncertainty, and no live SIEM/network data. It should therefore be treated as an academic threat-classification prototype rather than a production detection system.

Possible improvements: add event timestamps, severity levels, asset criticality, rule priorities, confidence scores, temporal correlation, real log ingestion, and a web/SOC dashboard.

10. Conclusion, References & GitHub

This modelling exercise represented the same cybersecurity threat-detection problem using production rules, propositional logic, first-order logic and Prolog. Forward chaining demonstrated data-driven rule firing, while backward chaining demonstrated goal-directed proof. The comparison shows that Prolog provides the strongest balance of expressiveness, reusable rule representation and executable inference for this problem.

References:

SIMATS Engineering, CO5 Assessment Tool 3: Modelling Comparative Analysis Assignment, Artificial Intelligence and Expert Systems, Course Code MLA01.

National Institute of Standards and Technology (NIST), The NIST Cybersecurity Framework (CSF) 2.0, NIST CSWP 29, 2024.

SWI-Prolog, SWI-Prolog Documentation and Reference Manual.

Compulsory Comparative Table:

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation				
Expressiveness				
Rule Representation				
Inference Mechanism				
Forward Chaining				
Backward Chaining				
Handling Relationships				
Scalability				
Explainability				
Ease of Implementation				
Suitability for the Problem				

EVALUATION RUBRICS

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, constraints, facts, and expected outcomes.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear or poorly understood.
2. Domain Knowledge and Fact Identification	10%	9–10% – Identifies comprehensive and relevant facts, entities, relationships, conditions, and domain knowledge.	7–8% – Identifies most relevant knowledge with minor gaps.	5–6% – Basic facts and entities are identified.	0–4% – Important domain knowledge is missing or incorrect.
3. Production Rule Modelling	15%	13–15% – Develops accurate, complete, consistent IF–THEN production rules that effectively represent the problem.	10–12% – Rules are appropriate with minor errors or omissions.	7–9% – Basic rules are developed but several are incomplete or unclear.	0–6% – Rules are largely incorrect, incomplete, or inappropriate.
4. Propositional Logic Modelling	10%	9–10% – Correctly represents domain knowledge using propositions, logical connectives, and valid inference statements.	7–8% – Good propositional representation with minor errors.	5–6% – Basic representation with limited logical formulation.	0–4% – Representation is incorrect or incomplete.

5. First-Order Logic Modelling	15%	13–15% – Accurately models entities, relationships, properties, variables, predicates, and quantifiers with valid FOL statements.	10–12% – Good FOL model with minor errors or omissions.	7–9% – Basic FOL representation with limited relationships or incorrect usage of some constructs.	0–6% – FOL model is largely incorrect or missing.
6. Prolog Modelling and Implementation	15%	13–15% – Successfully converts the knowledge model into Prolog facts, rules, predicates, and queries with correct outputs.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works but has limited functionality.	0–6% – Implementation is incomplete or non-functional.
7. Forward and Backward Chaining Demonstration	10%	9–10% – Clearly demonstrates both forward and backward chaining with correct inference sequences and conclusions.	7–8% – Both mechanisms are demonstrated with minor gaps.	5–6% – Basic demonstration with limited reasoning explanation.	0–4% – Chaining is absent or incorrectly demonstrated.
8. Comparative Analysis of Modelling Approaches	10%	9–10% – Provides a detailed and insightful comparison based on expressiveness, inference, complexity, scalability, flexibility, and explainability.	7–8% – Provides a meaningful comparison using appropriate parameters.	5–6% – Basic comparison with limited criteria or analysis.	0–4% – Little or no meaningful comparison.

9. Results, Discussion and Model Selection	3%	3% – Clearly analyzes results and convincingly justifies the most suitable approach for the selected problem.	2% – Provides good analysis and reasonable justification .	1% – Basic discussion with limited justification.	0% – No meaningful analysis or justification.
10. Documentation , GitHub Repository and Presentation	2%	2% – Report is complete and well structured, GitHub repository is organized, and presentation is professional.	1.5% – Good documentat ion and presentatio n with minor issues.	1% – Basic documentation with some omissions.	0% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				